

Подзапросы и общие табличные выражения

SQL & БАЗЫ ДАННЫХ

НЕДЕЛЯ 5

План занятия



→ Виды подзапросов

→ Применение подзапросов

→ Общие табличные выражения

Подзапрос

Что это?



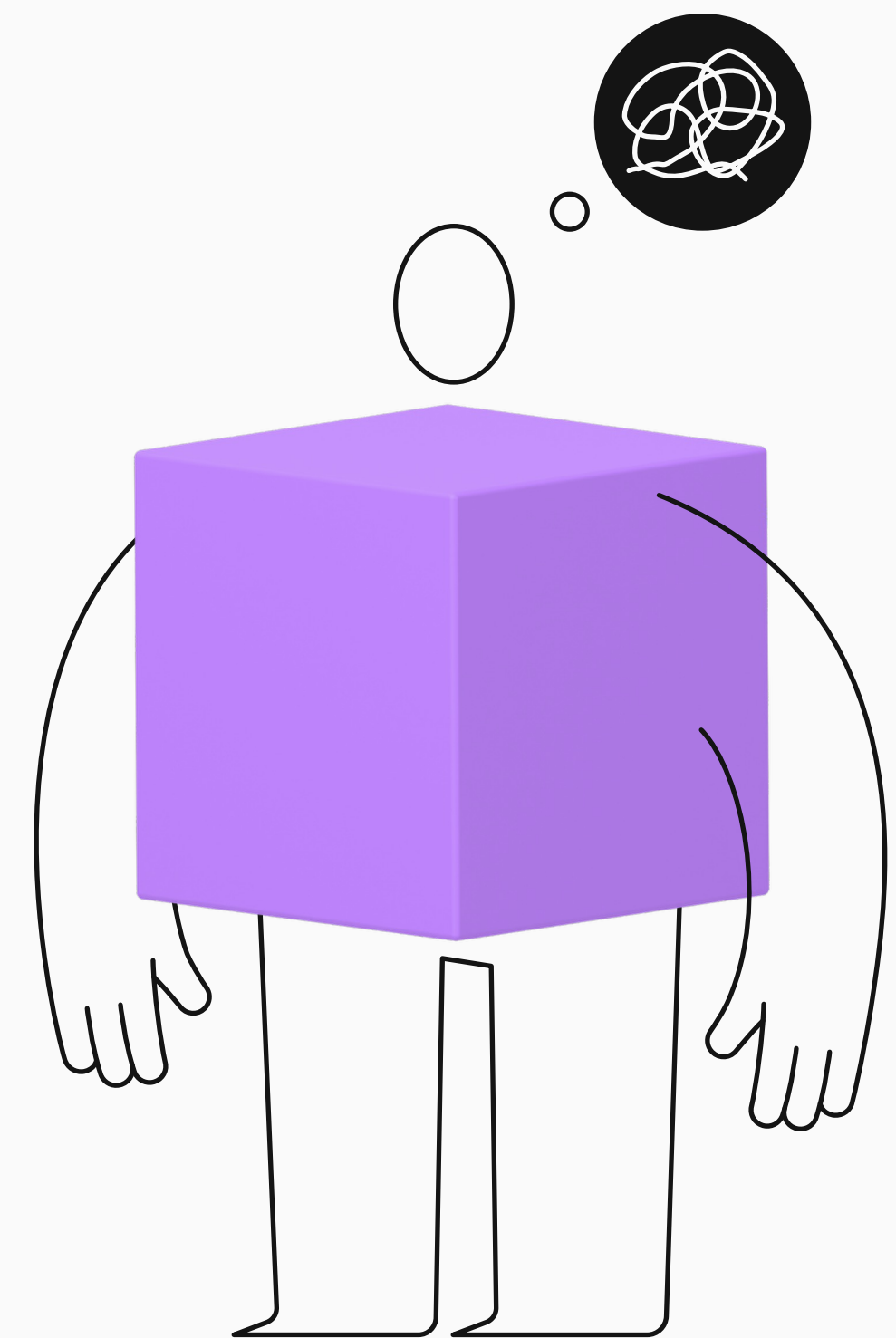
Запрос, который помещается внутрь другого SQL-запроса и чей результат используется основным (внешним) запросом для фильтрации данных, определения условий или в качестве источника данных.

Зачем?



Подзапросы позволяют строить более сложные и лаконичные запросы, решающие задачи, которые сложно или невозможно выполнить одним оператором.

Какими бывают подзапросы?



По связи с внешним запросом

Некоррелированные

- Не используют колонки внешнего запроса.
- Выполняются логически один раз, результат подставляется во внешний запрос.

```
SELECT name
FROM w_country
WHERE population > (
    SELECT AVG(population)
    FROM w_country
);
```

Коррелированные

- Ссылаются на колонки внешнего запроса ⇒ выполняются для каждой строки внешнего набора.
- Корреляция достигается за счёт внешних ссылок в тексте подзапроса:
ci.country_code = c.code.

```
SELECT c.name
FROM w_country c
WHERE c.population > (
    SELECT
        COALESCE(AVG(ci.population), 0)
    FROM w_city ci
    WHERE ci.country_code = c.code
);
```

По количеству возвращаемых строк и столбцов

Скалярные

- Ровно 1 строка и 1 столбец = 1 значение. Используются как обычное выражение.
- Если вернёт 0 строк → результат NULL.
- Если >1 строки → ошибка.
- Чтобы гарантировано получить одну строку, можно использовать Limit 1.

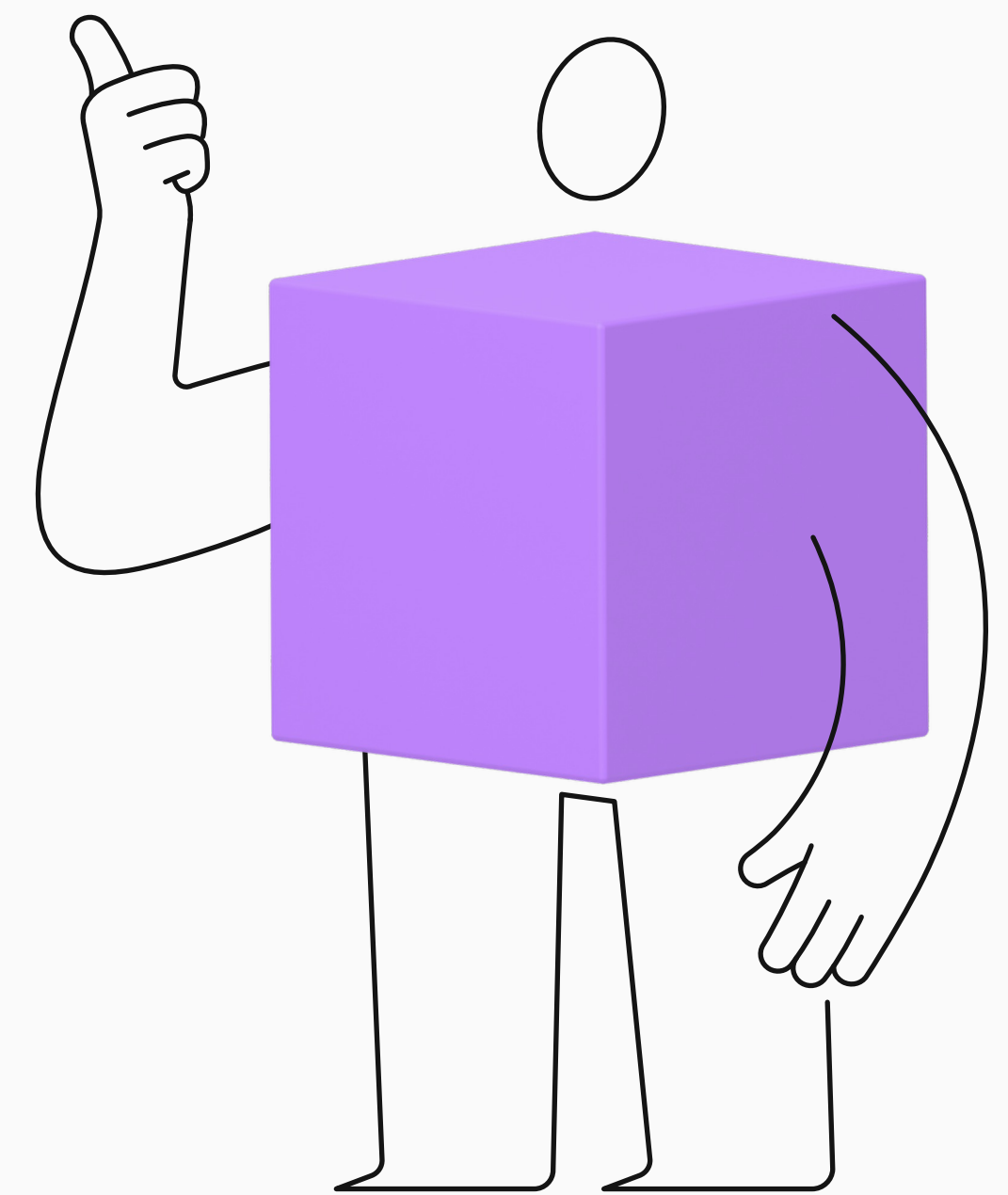
Списки

- 1 столбец, N строк. Используются с IN, ANY/SOME, ALL.

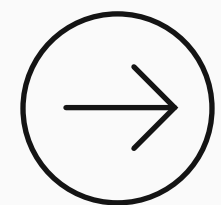
Табличные

- Много столбцов, много строк. Используются в FROM (подзапрос как временная таблица/вью).

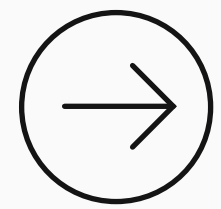
Подзапросы в разных частях запроса



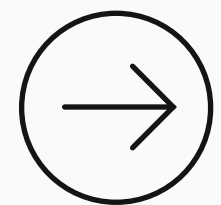
Где можно писать подзапросы в SQL?



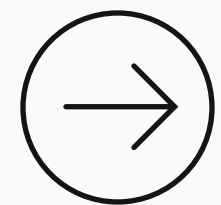
SELECT



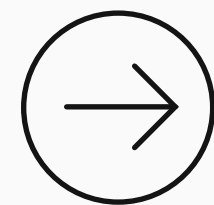
FROM



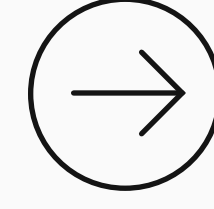
WHERE



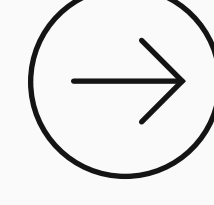
GROUP BY



HAVING

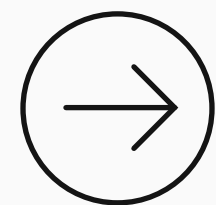


ORDER BY



LIMIT

Где можно писать подзапросы в SQL?

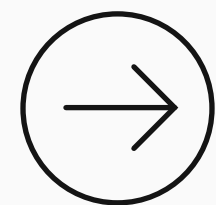


SELECT

- Скалярные (метрики/флаги в колонках)
- Коррелированные и некоррелированные

```
SELECT
    c.name
    , (
        SELECT COUNT(*)
        FROM w_city ci
        WHERE ci.country_code = c.code
    ) AS city_cnt
FROM w_country c;
```

Где можно писать подзапросы в SQL?

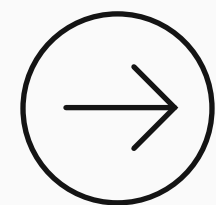


FROM

- Табличные, CTE (WITH), реже — скалярный
- Только некоррелированные, корреляция тут невозможна

```
SELECT s.product_id, s.rev
FROM (
    SELECT
        product_id, SUM(amount) AS rev
    FROM shop_orders
    GROUP BY product_id
) s;
```

Где можно писать подзапросы в SQL?



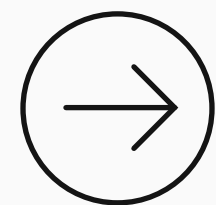
WHERE

- Скалярные
- Списки (**IN, ANY, ALL**)
- Коррелированные и некоррелированные

Про подзапросы Where подробнее будет дальше.

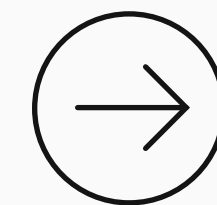
```
SELECT name
FROM w_country
WHERE population > (
    SELECT AVG(population)
    FROM w_country
);
```


Где можно писать подзапросы в SQL?



GROUP BY

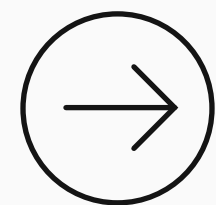
- Допускаются скалярные (очень редко используются)
- Некоррелированные и коррелированные



HAVING

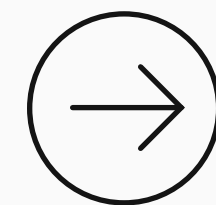
- Скалярные
- Списки для фильтрации групп после агрегации (**ANY, ALL**)
- Коррелированные и некоррелированные

Где можно писать подзапросы в SQL?



ORDER BY

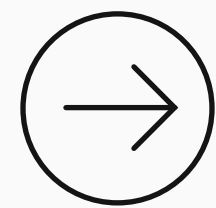
- Скалярные подзапросы (сортировка по вычисляемой метрике, используется редко);
- Коррелированные и некоррелированные



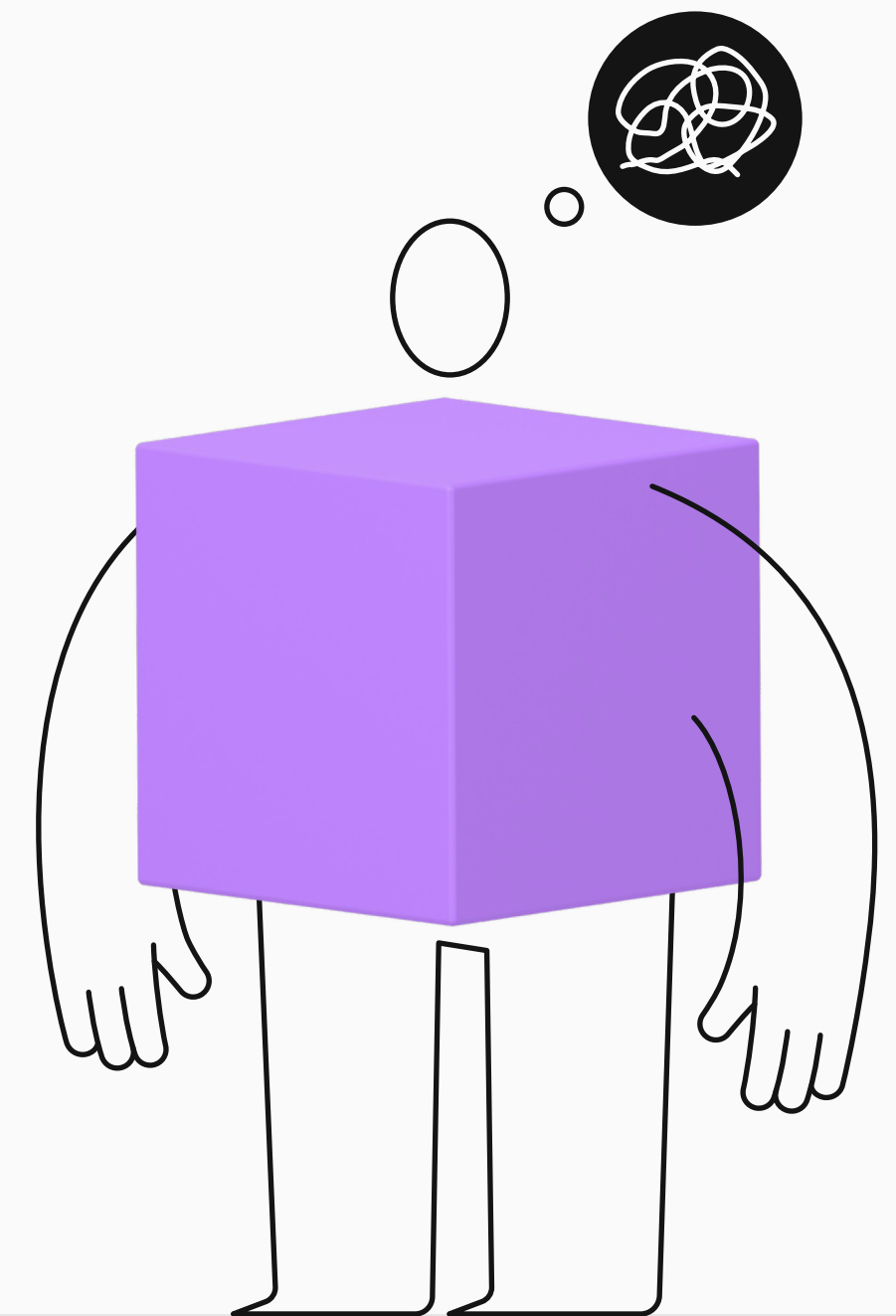
LIMIT

- Скалярные (динамический лимит, используется редко); поддерживается в PostgreSQL
- Некоррелированные

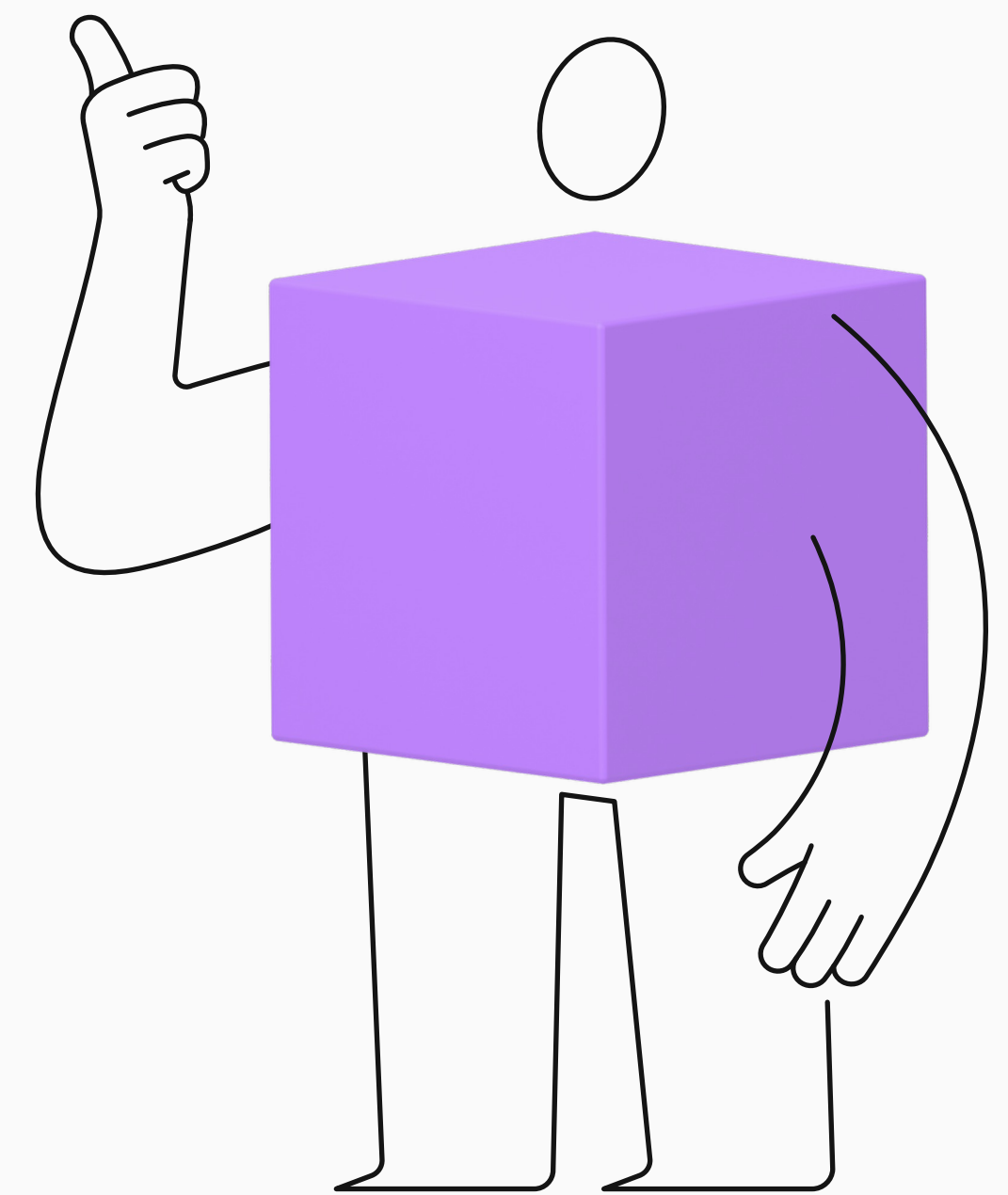
Где можно писать подзапросы в SQL?



Скалярный подзапрос можно писать везде, где ожидается выражение.



Фильтрация с использованием подзапросов



Зачем нужна фильтрация с использованием подзапросов

(01)

Выбор данных, релевантных
конкретному пользователю,
его настройкам
или контексту

Показывать события,
которые соответствуют
интересам пользователя

(02)

Сравнение каждой
строки с агрегированным
значением по подмножеству
данных

Сотрудники с зарплатой выше
средней по отделу

(03)

Учитывать текущее
состояние объектов
в связанной таблице

Клиенты, у которых последний
заказ был более 3 месяцев назад

(04)

Проверить, входит
ли значение из текущей
строки в множество,
полученное динамически

Заказы, в которых участвовали
товары из топ-10 категорий
клиента

Использование подзапросов в WHERE, HAVING

В SQL можно сформировать часть условия поиска с помощью запроса.

Например, в выражении $A > X$ или $A = X$, X можно вычислить с помощью запроса.

В выражении вида $X \text{ in } (\text{ListA})$ список значений ListA может быть сформирован с помощью запроса.

Использование подзапросов в WHERE, HAVING

В SQL можно сформировать часть условия поиска с помощью запроса.

Например, в выражении $A > X$ или $A = X$, X можно вычислить с помощью запроса.

В выражении вида $X \text{ in } (\text{ListA})$ список значений ListA может быть сформирован с помощью запроса.

```
SELECT столбец1, ...  
FROM таблица  
WHERE (Подзапрос)  
+ если нужно столбцы для сравнения
```

```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 = (Подзапрос)
```

```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 IN (Подзапрос)
```


Использование подзапросов в WHERE, HAVING

В SQL можно сформировать часть условия поиска с помощью запроса.

Например, в выражении $A > X$ или $A = X$, X можно вычислить с помощью запроса.

В выражении вида $X \text{ in } (\text{ListA})$ список значений ListA может быть сформирован с помощью запроса.

```
SELECT столбец1, ...  
FROM таблица  
WHERE (Подзапрос)  
+ если нужно столбцы для сравнения
```

```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 = (Подзапрос)
```

```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 IN (Подзапрос)
```

Использование подзапросов в WHERE, HAVING

В SQL можно сформировать часть условия поиска с помощью запроса.

Например, в выражении $A > X$ или $A = X$, X можно вычислить с помощью запроса.

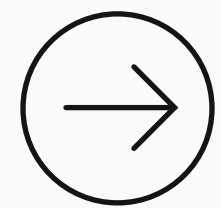
В выражении вида $X \text{ in } (\text{ListA})$ список значений ListA может быть сформирован с помощью запроса.

```
SELECT столбец1, ...  
FROM таблица  
WHERE (Подзапрос)  
+ если нужно столбцы для сравнения
```

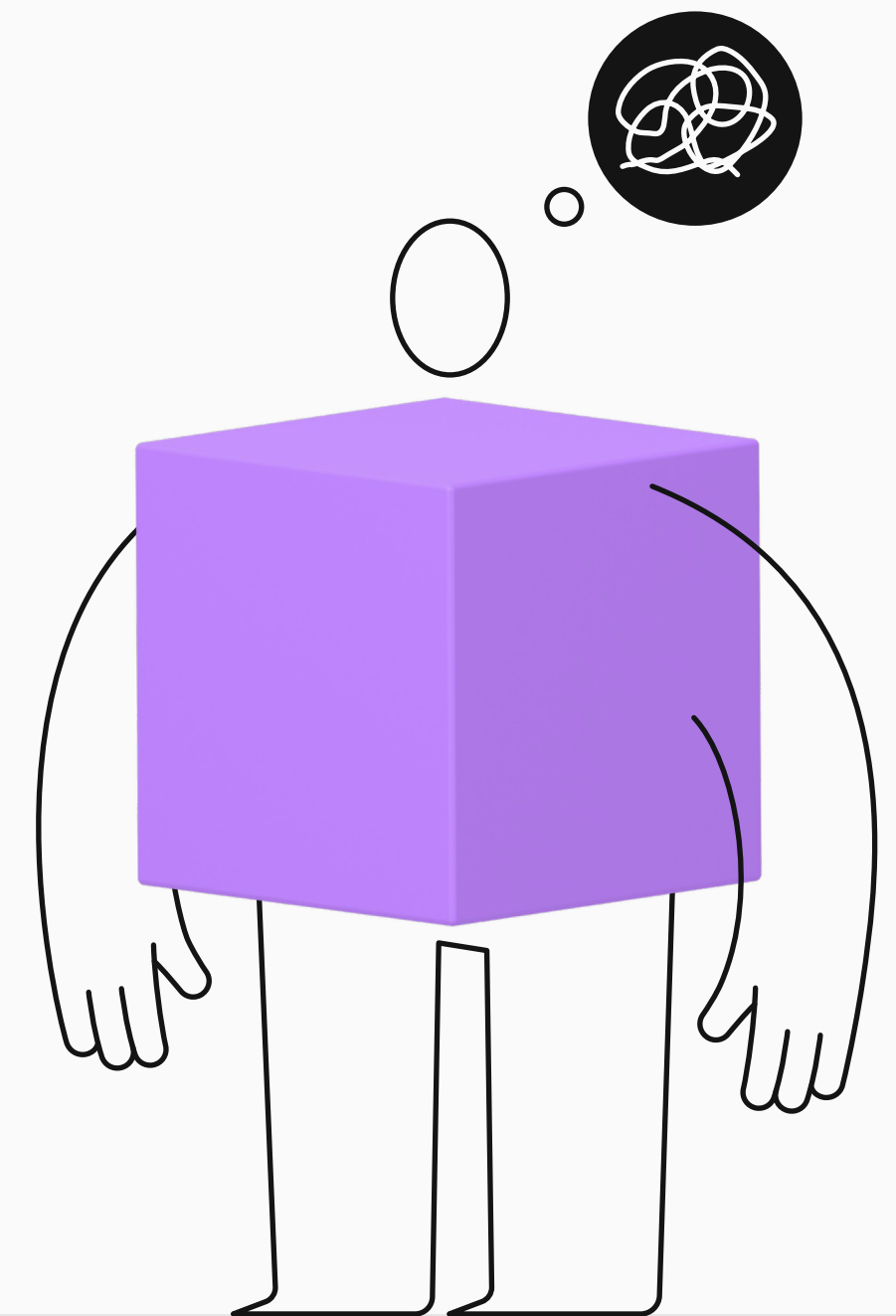
```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 = (Подзапрос)
```

```
SELECT столбец1, ...  
FROM таблица  
WHERE столбец1 IN (Подзапрос)
```

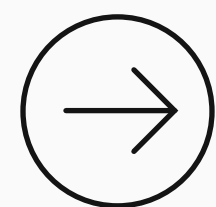
Скалярный подзапрос



Возвращает ? столбец и ? строку (? значение).

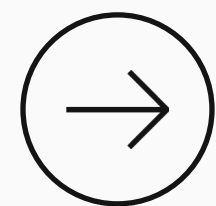


Скалярный подзапрос



Возвращает 1 столбец и 1 строку (1 значение).

```
SELECT столбец1 -- только 1 столбец
FROM таблица1 т1
WHERE условия -- если нужны
GROUP BY ключ агрегации -- если нужен
HAVING условия -- если нужны
ORDER BY правило сортировки -- если нужен
LIMIT 1 -- гарантия, что строка будет 1
```



Используется со следующими операциями сравнения:
(с какими операциями?) + (скалярный подзапрос).

Скалярный подзапрос

- Возвращает 1 столбец и 1 строку (1 значение).
- Используется со следующими операциями сравнения:
=, >, <, !=, >=, <=, **like** + (скалярный подзапрос).

```
SELECT столбец1, ...  
FROM таблица1 т1  
WHERE столбецX = (  
    SELECT столбец1  
    FROM таблица2  
    LIMIT 1)
```

```
SELECT столбец1, ...  
FROM таблица1 т1  
WHERE столбецX LIKE (  
    SELECT столбец1  
    FROM таблица2  
    LIMIT 1)
```

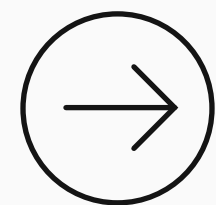
- Для фильтрации можно использовать в каких предложениях?

Скалярный подзапрос

- Возвращает 1 столбец и 1 строку (1 значение).
- Используется со следующими операциями сравнения: `=`, `>`, `<`, `!=`, `>=`, `<=`, `like` + (скалярный подзапрос).
- Для фильтрации можно использовать как в **WHERE**, так и в **HAVING**.

```
SELECT столбец1, ...  
FROM таблица1 т1  
WHERE столбецX = (  
    SELECT столбец1  
    FROM таблица2  
    LIMIT 1)
```

```
SELECT столбец1, avg(столбец2), ...  
FROM таблица1 т1  
GROUP BY столбец1, ...  
HAVING avg(столбецX) > (  
    SELECT столбец1  
    FROM таблица2  
    LIMIT 1)
```



Фильтрация с помощью скалярного запроса

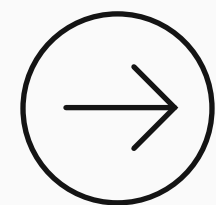
Нужно найти страны с площадью, превышающей 80% от площади самой большой страны мира.

Что требуется в ответе?

Ответ должен содержать колонки:

1. region — регион страны;
2. name — название страны;
3. population — население страны.





Фильтрация с помощью скалярного запроса

Нужно найти страны с площадью, превышающей 80% от площади самой большой страны мира.

Что требуется в ответе?

Ответ должен содержать колонки:

1. region — регион страны;
2. name — название страны;
3. population — население страны.

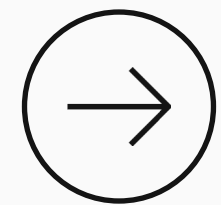
```
SELECT
    region,
    name,
    population
FROM w_country AS c
WHERE
    surface_area > 0.8 * (
        SELECT max(surface_area)
        FROM w_country
    );
```

Скалярный подзапрос.
Вернёт 1 строку, 1 столбец.

Динамическая фильтрация

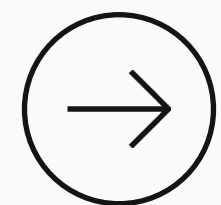
- Это использование коррелированного подзапроса в условии фильтра в **WHERE** или **HAVING**.
- Динамическое условие поиска может быть разным для различных строк.
- Вычисления будут выполняться несколько раз, в худшем случае — для каждой строки внешнего запроса.

Динамическая фильтрация

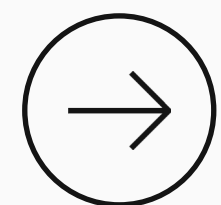


Это использование коррелированного подзапроса в условии фильтра в **WHERE** или **HAVING**.

```
SELECT столбец1, ...  
FROM таблица1 т1  
WHERE столбец1 = (  
    SELECT т2.столбец1  
    FROM таблица2 т2  
    WHERE т1.столбецN = т2.столбецM) ← Связь подзапроса с внешним запросом
```



Динамическое условие поиска может быть разным для различных строк.



Вычисления будут выполняться несколько раз,
в худшем случае — для каждой строки внешнего запроса.

Динамическая фильтрация

- Это использование коррелированного подзапроса в условии фильтра в **WHERE** или **HAVING**.
- Динамическое условие поиска может быть разным для различных строк.

```
SELECT столбец1, ...  
FROM таблица1 т1  
WHERE столбец1 = (коррелированный подзапрос)
```

Результат коррелированного подзапроса:

для строки 1 — это 5;

для строки 2 — это 7;

...

для строки N — это 4.

- Вычисления будут выполняться несколько раз,
в худшем случае — для каждой строки внешнего запроса.

Динамическая фильтрация

- Это использование коррелированного подзапроса в условии фильтра в **WHERE** или **HAVING**.
 - Динамическое условие поиска может быть разным для различных строк.
-
- Вычисления будут выполняться несколько раз, в худшем случае — для каждой строки внешнего запроса.

Найти регионы с наибольшей численностью населения на континенте.

Формат вывода:

- название континента,
- название региона,
- площадь региона.

Надо вывести все регионы с максимальной численностью населения на континенте.



Найти регионы с наибольшей численностью населения на континенте.

1. Найти численность населения регионов.
2. Найти численность населения регионов на конкретном континенте.
3. Найти максимальную численность населения среди регионов.
4. Вывести все регионы с этой численностью. Поместить запрос из шага 3 в having. Частный случай (Европа)
5. Сделать запрос коррелированным.

```
SELECT
    sum(population) as p
FROM w_country
GROUP BY region
```

ДЕМО

Найти регионы с наибольшей численностью населения на континенте.

1. Найти численность населения регионов.
2. Найти численность населения регионов на конкретном континенте.
3. Найти максимальную численность населения среди регионов.
4. Вывести все регионы с этой численностью. Поместить запрос из шага 3 в having. Частный случай (Европа)
5. Сделать запрос коррелированным.

```
SELECT
    sum(population) AS p
FROM w_country
WHERE continent = 'Europe'
GROUP BY region
```

Найти регионы с наибольшей численностью населения на континенте.

1. Найти численность населения регионов.
2. Найти численность населения регионов на конкретном континенте.
3. Найти максимальную численность населения среди регионов.
4. Вывести все регионы с этой численностью. Поместить запрос из шага 3 в having. Частный случай (Европа)
5. Сделать запрос коррелированным.

```
SELECT max(p)
FROM (
    SELECT sum(population) AS p
    FROM w_country
    WHERE continent = 'Europe'
    GROUP BY region
) population_region
```


ДЕМО

Найти регионы с наибольшей численностью населения на континенте.

1. Найти численность населения регионов.
2. Найти численность населения регионов на конкретном континенте.
3. Найти максимальную численность населения среди регионов.
4. Вывести все регионы с этой численностью.
Поместить запрос из шага 3 в `having`.
Частный случай (Европа)
5. Сделать запрос коррелированным.

```
SELECT
    continent, region,
    sum(surface_area) AS region_s_a
FROM w_country c
GROUP BY region, continent
HAVING sum(population) = (
    SELECT max(p)
    FROM (
        SELECT sum(population) AS p
        FROM w_country
        WHERE continent = 'Europe'
        GROUP BY region
    ) population_region);
```

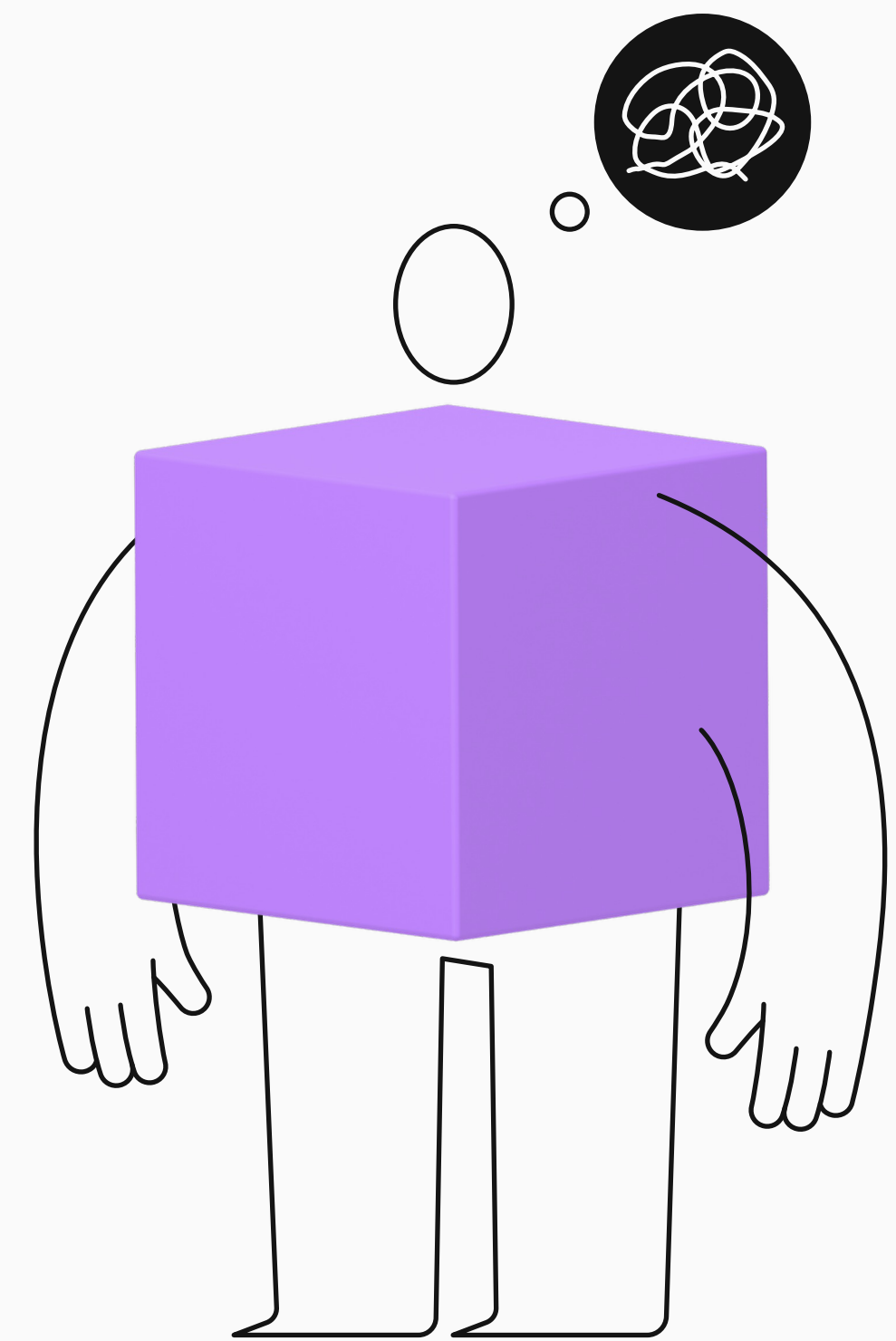
5. Сделать запрос коррелированным.

Коррелированный подзапрос
в **HAVING** возвращает
наибольшее количество жителей
региона в конкретном континенте.

Делает подзапрос коррелированным.

```
SELECT
    continent,
    region,
    sum(surface_area) AS region_s_a
FROM w_country c
GROUP BY region, continent
HAVING sum(population) = (
    SELECT max(p)
    FROM (
        SELECT sum(population) AS p
        FROM w_country
        WHERE continent = c.continent
        GROUP BY region) population_region
```

**Зачем нужна фильтрация
с использованием
подзапросов со списками?**



Зачем нужна фильтрация с использованием подзапросов со списками

(01)

**Фильтрация по связанным
сущностям**

Найти все города,
которые находятся
в странах Европы

(02)

**Поиск по условиям
из другой таблицы**

Показать страны,
в которых говорят
по-испански

(03)

**Поиск по результатам
агрегации**

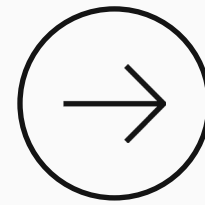
Найти страны,
в которых суммарное
население всех городов
превышает 25 миллионов

(04)

**Для этих задач скалярного
подзапроса недостаточно**

Подзапросы в WHERE

Фильтрация строк
основного запроса



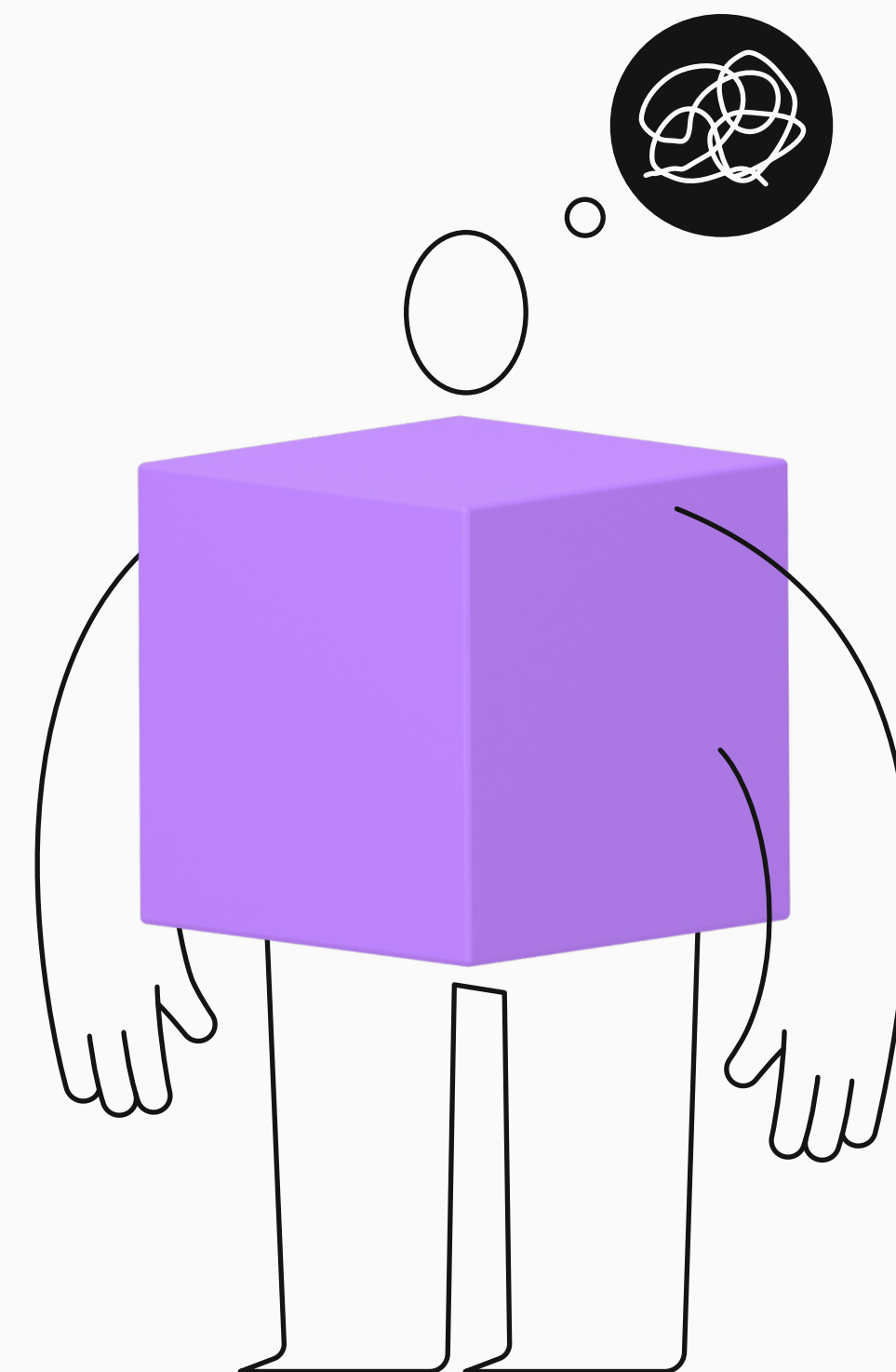
Синтаксис

```
SELECT столбец1, столбец2
FROM таблица1
WHERE столбец_название {оператор} (
    SELECT столбец_название
    FROM таблица2
    WHERE условие
    ...
)
...
```

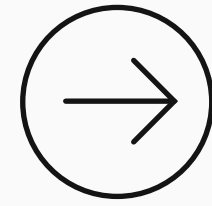
Подзапрос должен возвращать один столбец.



Подзапросы в WHERE.
Какие операторы
вы знаете?

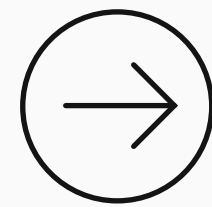


Подзапросы В WHERE



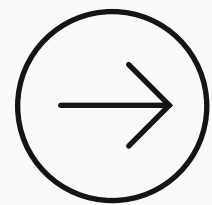
IN / NOT IN

Принадлежность множеству значений из подзапроса.



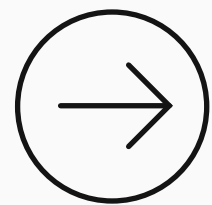
ANY / SOME

Проверка соответствия хотя бы одному значению подзапроса.



ALL

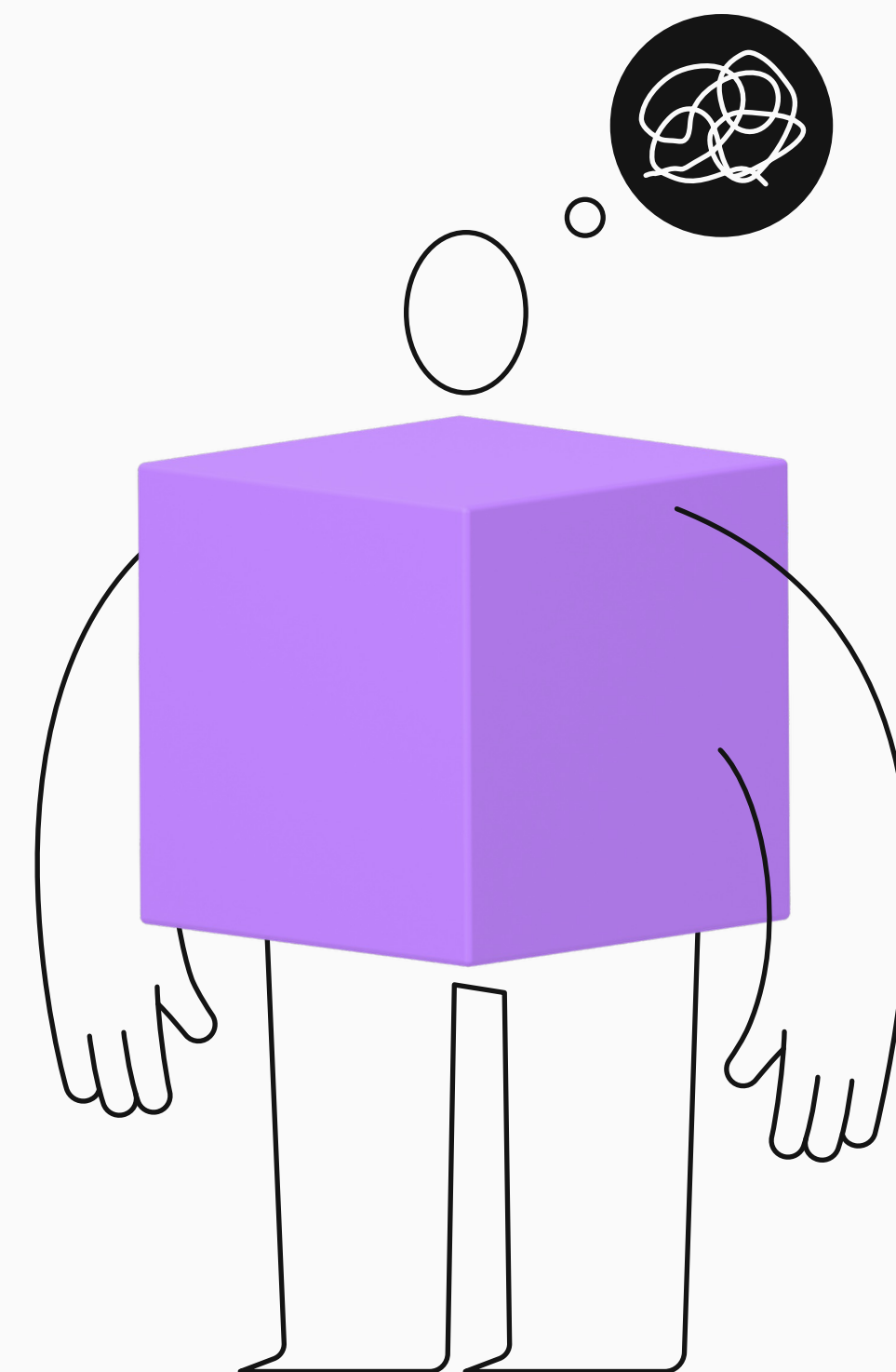
Проверка соответствия всем значениям подзапроса.



EXISTS

Проверка существования строк в подзапросе.

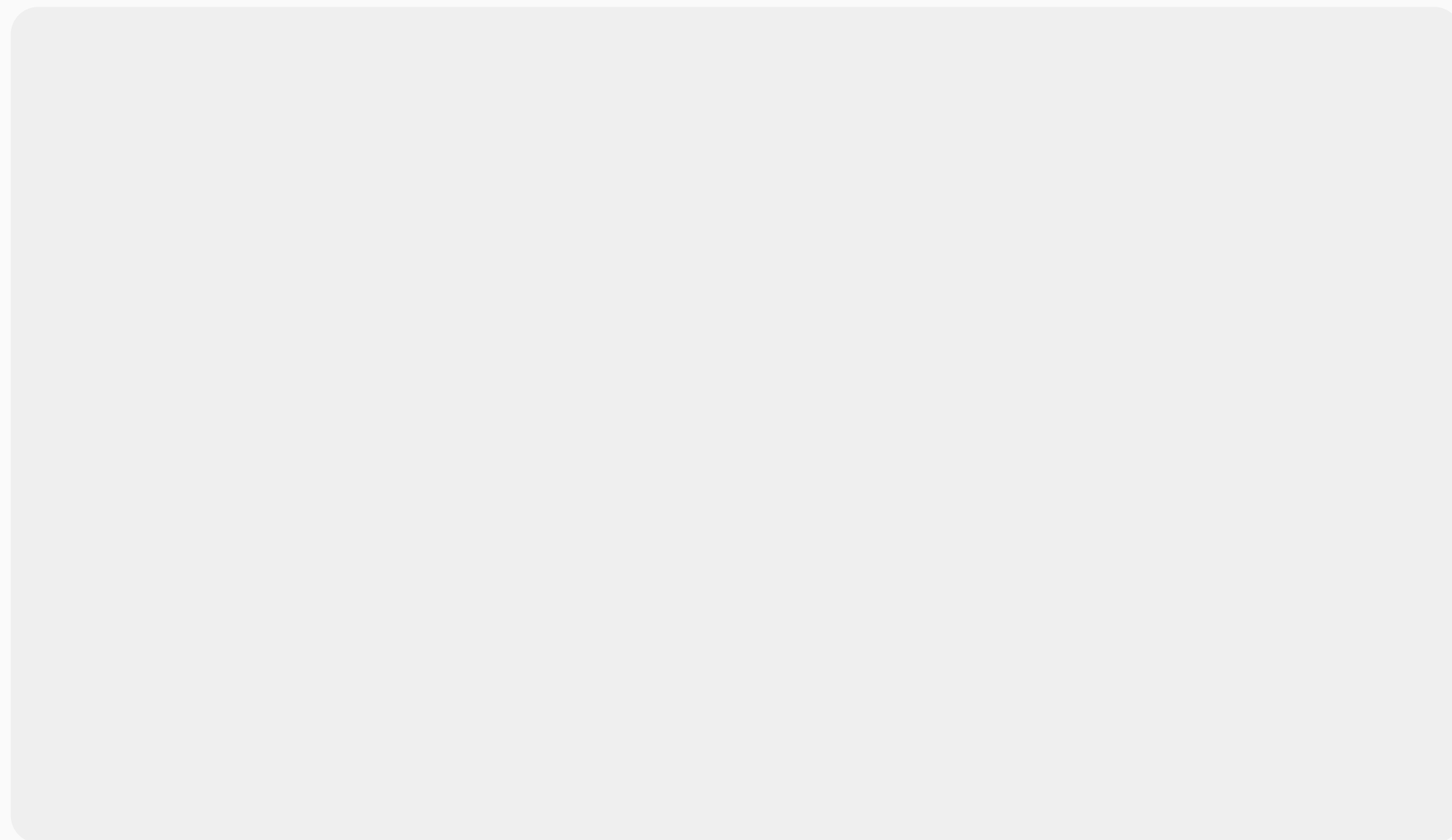
SEMI JOIN и ANTI JOIN. Что это за операции?



SEMI JOIN



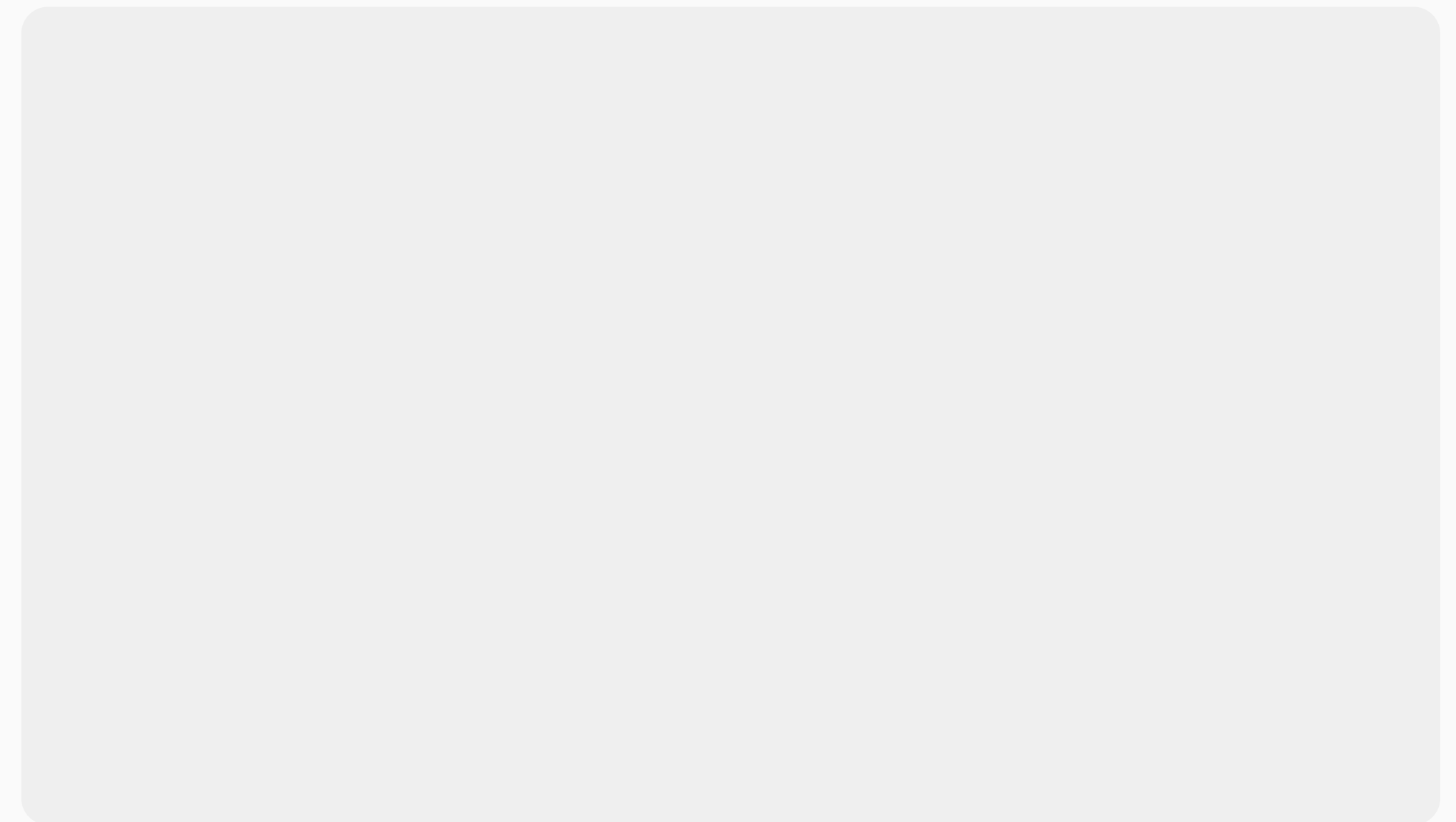
Это операция, при которой выбираются строки из одной таблицы, если для них существует хотя бы одна подходящая строка во второй таблице, но данные из второй таблицы не возвращаются.



ANTI JOIN



Это операция, при которой выбираются строки из одной таблицы, если для них не существует подходящей строки во второй таблице.





SEMI JOIN

Это операция, при которой выбираются строки из одной таблицы, если для них существует хотя бы одна подходящая строка во второй таблице, но данные из второй таблицы не возвращаются.

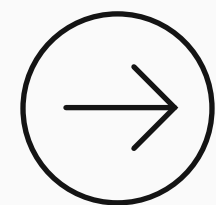
- **WHERE EXISTS**
- **WHERE IN**
- **INTERSECT**



ANTI JOIN

Это операция, при которой выбираются строки из одной таблицы, если для них не существует подходящей строки во второй таблице.

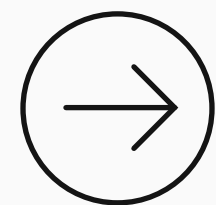
- **WHERE NOT EXISTS**
- **WHERE NOT IN**
- **EXCEPT**



Нужно найти города, которые являются столицами.

Можно реализовать этот запрос как минимум тремя способами.

1. С помощью **INNER JOIN**
2. С помощью подзапроса в **IN** (операция полусоединения, или **SEMI JOIN**)
3. С помощью предиката **EXISTS** (операция полусоединения, или **SEMI JOIN**)

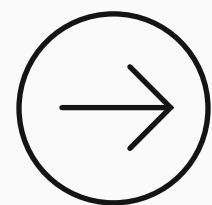


Нужно найти города, которые являются столицами.

1. С помощью `INNER JOIN`

```
SELECT
    ct.id,
    ct.name
FROM w_city ct
    INNER JOIN w_country wc
        ON ct.id = wc.capital
```

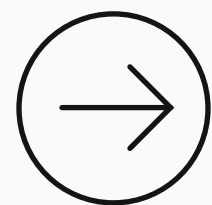
Полноценный `join`,
в котором мы сначала
соединили две таблицы
в 1 и далее сделали
проекцию (оставили
две колонки).



Нужно найти города, которые являются столицами.

Можно реализовать этот запрос как минимум тремя способами.

1. С помощью **INNER JOIN**
2. С помощью подзапроса в **IN** (операция полусоединения, или **SEMI JOIN**)
3. С помощью предиката **EXISTS** (операция полусоединения, или **SEMI JOIN**)

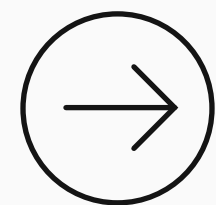


Нужно найти города, которые являются столицами.

2. С помощью подзапроса в IN (операция полусоединения, или SEMI JOIN)

```
SELECT
    ct.id,
    ct.name
FROM w_city ct
WHERE ct.id IN (
    SELECT capital
    FROM w_country wc
)
```

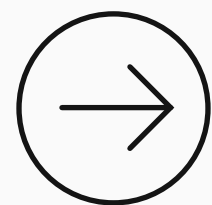
Проверяем, входит ли **id** города в список **id** столиц из таблицы стран. Корреляция не нужна. Нет необходимости присоединять в **from** столбцы из таблицы стран.



Нужно найти города, которые являются столицами.

Можно реализовать этот запрос как минимум тремя способами.

1. С помощью **INNER JOIN**
2. С помощью подзапроса в **IN** (операция полусоединения, или **SEMI JOIN**)
3. С помощью предиката **EXISTS** (операция полусоединения, или **SEMI JOIN**)



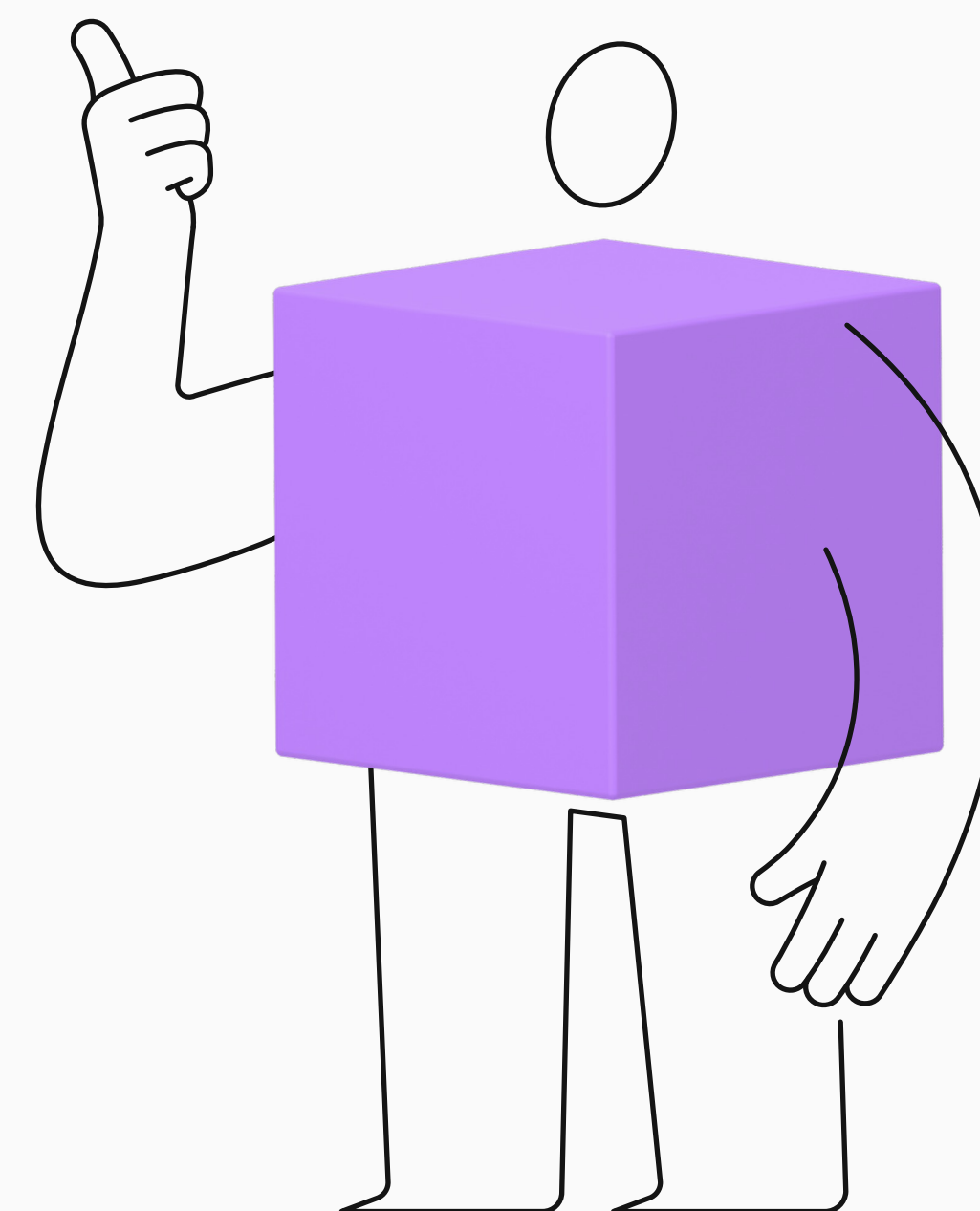
Нужно найти города, которые являются столицами.

3. С помощью предиката **EXISTS** (операция полусоединения, или **SEMI JOIN**)

```
SELECT
    ct.id,
    ct.name
FROM w_city ct
WHERE EXISTS (
    SELECT 1
    FROM w_country c
    WHERE c.capital = ct.id
);
```

Проверяем, есть ли хотя бы 1 строка при попытке соединить **id** города с **id** столицы. Корреляция в **WHERE** нужна, без неё запрос всего будет возвращать **TRUE**. Нет необходимости присоединять в **FROM** столбцы из таблицы стран.

Общие табличные выражения





Общие табличные выражения

(Common Table Expressions, CTE) — это временные результирующие наборы, которые можно использовать в рамках одного запроса.

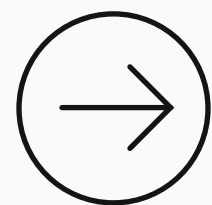
Общий синтаксис

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT ...  
FROM cte_name;
```

Использование нескольких СТЕ в одном запросе

Общие табличные выражения
необходимо объявлять
последовательно через запятую.

```
WITH first_CTE AS
(
    SELECT person_id, name
    FROM employee
    WHERE head_id IS NULL
)
, second_CTE AS
(
    SELECT person_id, name
    FROM employee
    WHERE head_id IS NOT NULL
)
SELECT * FROM first_CTE
UNION ALL
SELECT * FROM second_CTE;
```

Фильтрация с помощью запроса

Выявить категории продуктов, чья общая сумма продаж выше средней суммы продаж по всем категориям. Использование **СТЕ**.

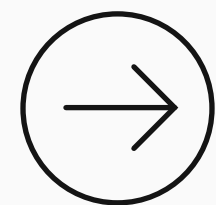
Задача: используя **СТЕ** для расчёта суммы продаж по категориям, напишите запрос, который выводит

* **category_id**,

* **total_sales** для категорий с продажами выше среднего.

```
SELECT
    p.category_id,
    SUM(o.total_amount) AS total_sales
FROM shop_products p
    INNER JOIN shop_orderitems oi
        ON p.product_id = oi.product_id
    INNER JOIN shop_orders o
        ON oi.order_id = o.order_id
GROUP BY p.category_id
```

Запрос справа можно использовать как **СТЕ** для решения задачи.



Фильтрация с помощью запроса

Выявить категории продуктов, чья общая сумма продаж выше средней суммы продаж по всем категориям. Использование **CTE**.

```
WITH category_sales AS (  
    SELECT  
        p.category_id,  
        SUM(o.total_amount) AS total_sales  
    FROM shop_products p  
        INNER JOIN shop_orderitems oi  
            ON p.product_id = oi.product_id  
        INNER JOIN shop_orders o  
            ON oi.order_id = o.order_id  
    GROUP BY p.category_id  
)  
SELECT  
    category_id,  
    total_sales  
FROM category_sales  
WHERE total_sales > (  
    SELECT AVG(total_sales) FROM category_sales  
) ;
```

CTE можно использовать и в подзапросе для фильтрации.

Рекурсивные СТЕ

Определение

Позволяют выполнять итеративные вычисления.
Например, организовывать иерархические структуры.
Они состоят из двух частей, которые соединяются оператором `union all`.



Первая часть

Начальный (якорный) запрос формирует начальный набор данных.



Вторая часть

Рекурсивный запрос ссылается на СТЕ и выполняется до тех пор, пока не будут обработаны все уровни иерархии.



ДЕМО 2

Необходимо определить уровень подчинённости всех сотрудников.

person_id	name	head_id
1	Елена	NULL
2	Кирилл	1
3	Андрей	2
4	Анастасия	1
5	Александр	4
6	Константин	4
7	Ольга	5
8	Мария	1
9	Владислав	7
10	Олег	4

ДЕМО 2

Необходимо определить уровень подчинённости всех сотрудников.

```
WITH RECURSIVE hierarchy AS (  
    SELECT id, name, head_id  
        , 1 AS level  
    FROM employee  
    WHERE head_id IS NULL  
    UNION ALL  
    SELECT e.id, e.name, e.head_id  
        , h.level + 1  
    FROM employee e  
        INNER JOIN hierarchy h  
            ON e.head_id = h.id  
    WHERE h.level < 3)  
SELECT *  
FROM hierarchy  
ORDER BY 1;
```

Условие `where h.level < 3` ограничивает глубину рекурсии до 3.

ДЕМО 2

Необходимо определить уровень подчинённости всех сотрудников.

person_id	name	head_id	level
1	Елена	NULL	1
2	Кирилл	1	2
3	Андрей	2	3
4	Анастасия	1	2
5	Александр	4	3
6	Константин	4	3
7	Ольга	5	4
8	Мария	1	2
9	Владислав	7	5
10	Олег	4	3

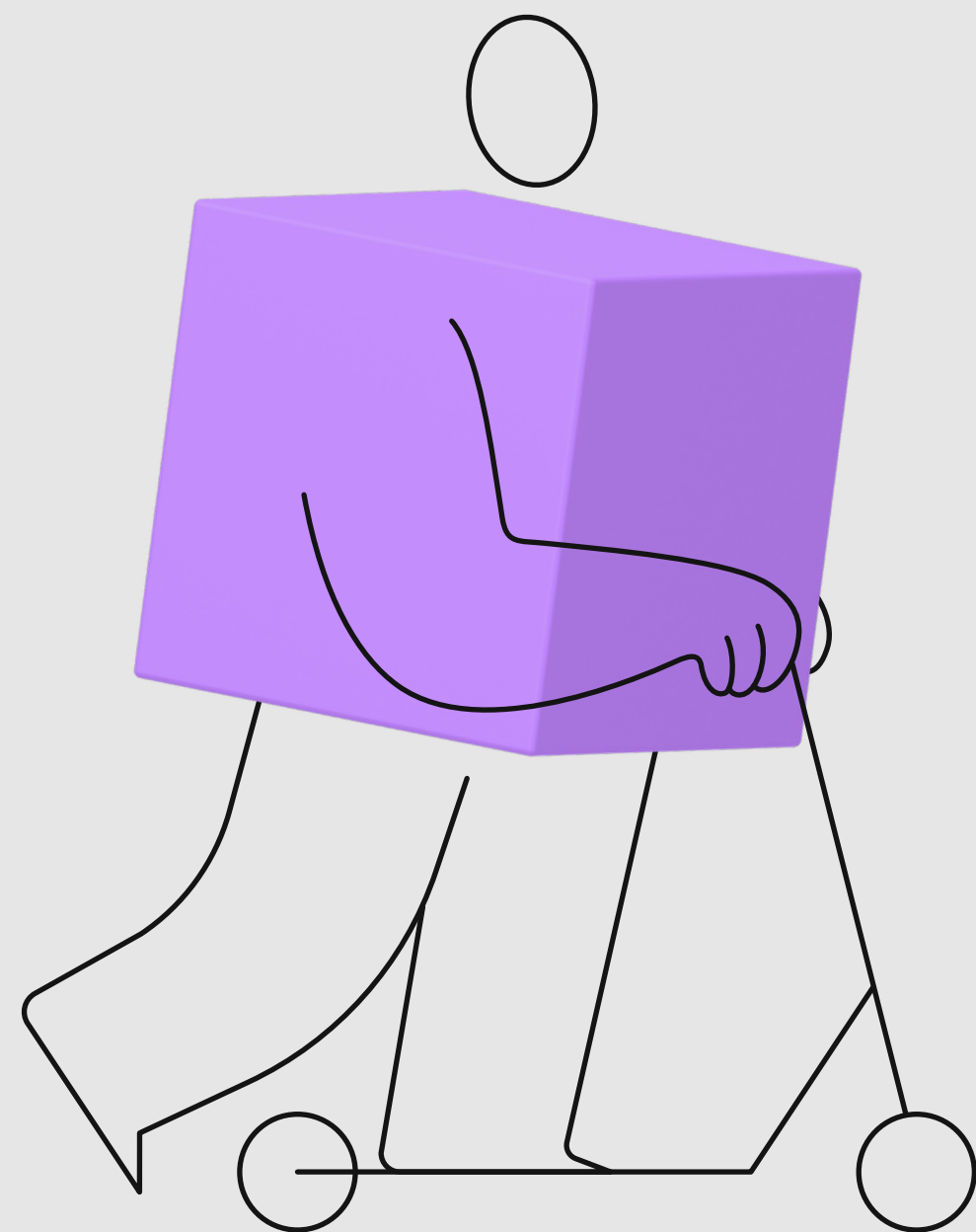
ДЕМО 2

Генерация последовательности чисел

```
WITH RECURSIVE numbers(n) AS (  
  SELECT 1  
  
  UNION ALL  
  
  SELECT n + 1  
  FROM numbers  
  WHERE n < 10  
)  
SELECT n  
FROM numbers;
```

n
1
2
3
4
5
6
7
8
9
10

Резюме



Подзапросы делают фильтрацию гибкой.

Ключевые практики:

- используй EXISTS/NOT EXISTS для проверки наличия/отсутствия (надёжно при NULL);
- для IN/NOT IN фильтруй NULL в подзапросах;
- явно приводи типы при сравнении разнородных колонок;
- помни про особенности ANY/ALL и трёхзначную логику SQL.

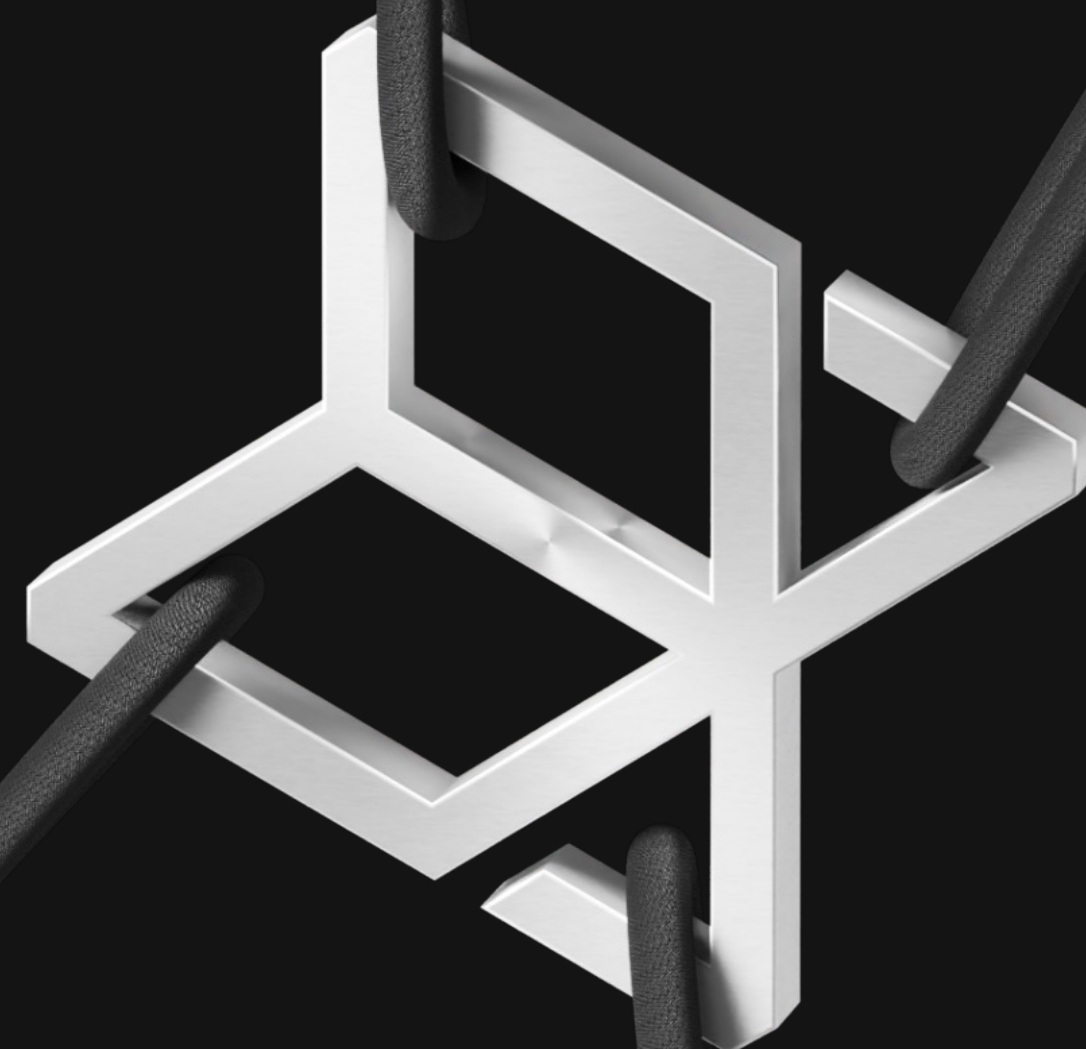


Подзапросы делают фильтрацию гибкой. CTE (WITH):

- применяй CTE для структурирования сложных запросов и повышения читаемости;
- используй WITH RECURSIVE для иерархий;



Вопросы



Минутка обратной связи

